Estruturas de dados

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Linguagem de Programação Voltada a Microcontroladores

Semana 02

Prof. Denis Moraes Guimarães

28/09/2026

Objetivos da aula

Ao final desta aula, você deverá ser capaz de:

- armazenar leituras em vetores e tabelas em matrizes;
- percorrer elementos sem ultrapassar seus limites;
- calcular soma, média, mínimo e máximo e fazer uma busca;
- representar texto como uma sequência de caracteres em C;
- comparar strings e ler texto com limite de capacidade.

Conexão com a aula anterior

Os laços permitem repetir operações. Agora, cada repetição pode trabalhar com uma posição diferente de um conjunto de dados.

Roteiro: de uma leitura a uma coleção

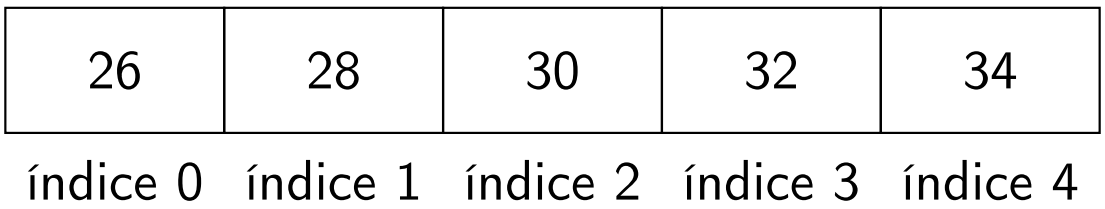
Estrutura	O que representa	Aplicação
Vetor	Sequência de elementos do mesmo tipo	Histórico de temperatura.
Matriz	Elementos organizados em linhas e colunas	Sensores em vários instantes.
String C	Caracteres terminados por zero	Comando recebido no terminal.

1. Declarar, inicializar e acessar.
2. Percorrer, atualizar, agregar e buscar.
3. Ler e comparar pequenos comandos de automação.

5.1 Vetores: elementos e capacidade

Um vetor reúne elementos do mesmo tipo em posições consecutivas.

```
int temperaturas[5] = {26, 28, 30, 32, 34};
```



- Capacidade: cinco elementos do tipo `int`.
- Primeiro índice: 0; último índice válido: 4.
- `temperaturas[2]` contém 30, o terceiro elemento.

Declaração e inicialização

```c
int leituras[4] = {10, 20, 30, 40};
int contadores[4] = {0}; /* todos recebem zero */
int alarmes[4] = {1, 2}; /* restante recebe zero */
int amostras[] = {7, 8, 9}; /* tamanho: 3 */
int pendentes[4]; /* locais: ainda sem valores */
```

- Um inicializador parcial preenche o restante com zero.
- Um vetor local sem inicialização deve receber valores antes de qualquer leitura.
- A capacidade de um vetor comum não aumenta durante o uso.

Convenção nesta aula

Usaremos tamanhos pequenos e constantes, para acompanhar cada acesso manualmente.

Índices: o limite é responsabilidade do programa

```c
int leituras[5] = {26, 28, 30, 32, 34};
int indice = 4;
if (indice >= 0 && indice < 5) {
    printf("Leitura: %d\n", leituras[indice]);
} else {
    printf("Indice invalido\n");
}
```

Acesso fora dos limites

`leituras[5]` e `leituras[-1]` são inválidos. Em C, esse acesso tem comportamento indefinido: não existe garantia de erro visível, proteção automática ou resultado previsível.

Para um vetor de capacidade N, o intervalo válido é $0 \leq i < N$.

Percorrer um vetor com for

```c
int leituras[5] = {26, 28, 30, 32, 34};
for (int i = 0; i < 5; i++) {
    printf("Sensor[%d] = %d C\n", i, leituras[i]);
}
```

i	Condição i < 5	Elemento
0	verdadeira	26
1	verdadeira	28
4	verdadeira	34
5	falsa	nenhum acesso

As posições 2 e 3 também são visitadas; a tabela destaca o início e o limite.

5.2 Operações básicas: consultar e atualizar

```c
int contagem[3] = {10, 20, 30};
contagem[1] = 25; /* substitui a segunda posicao */
contagem[2] += 1; /* incrementa a terceira */
for (int i = 0; i < 3; i++) {
    printf("%d\n", contagem[i]);
}
```

- Estado final: `{10, 25, 31}`.
- Atribuir a um elemento não modifica os demais.
- Consultar um elemento não o remove do vetor.

Antes de alterar uma posição

Valide qualquer índice vindo de uma entrada externa. Use a mesma capacidade na declaração e na condição do laço.

Soma e média: um acumulador para a coleção

```c
int leituras[5] = {26, 28, 30, 32, 34};
int soma = 0;
for (int i = 0; i < 5; i++) {
    soma += leituras[i];
}
double media = (double)soma / 5;
printf("Soma: %d; media: %.1f C\n", soma, media);
```

- Soma: 150; média: 30,0 °C.
- O acumulador começa em zero, antes do laço.
- A conversão para `double` evita divisão inteira.
- Na aplicação geral, verifique quantidade maior que zero e se a soma cabe no tipo escolhido.

Mínimo e máximo: começar com um elemento real

```
int minimo = leituras[0];
int maximo = leituras[0];
for (int i = 1; i < 5; i++) {
    if (leituras[i] < minimo) {
        minimo = leituras[i];
    }
    if (leituras[i] > maximo) {
        maximo = leituras[i];
    }
}
```

- Pressuposto: o vetor tem pelo menos um elemento válido.
- Com as leituras anteriores: mínimo 26 e máximo 34.
- Iniciar o mínimo em zero falha quando todas as leituras são positivas: zero não pertence à coleção.

Busca linear: localizar a primeira ocorrência

```c
int alvo = 30;
int encontrado = -1;
for (int i = 0; i < 5; i++) {
    if (leituras[i] == alvo) {
        encontrado = i;
        break;
    }
}
if (encontrado >= 0) {
    printf("Encontrado no indice %d\n", encontrado);
} else {
    printf("Valor ausente\n");
}
```

O valor -1 significa "não encontrado" e nunca deve ser usado como índice. A busca visita no máximo cinco elementos neste exemplo.

Rastreio da busca e da agregação

Vetor `{26, 28, 30, 32, 34}`; alvo 30:

Índice	Leitura	Soma acumulada	Comparação na busca
0	26	26	diferente
1	28	54	diferente
2	30	84	igual: parar a busca
3	32	116	não visitado na busca
4	34	150	não visitado na busca

São dois percursos com objetivos diferentes

A soma percorre todos os elementos. A busca da primeira ocorrência pode parar assim que encontra o alvo. Não use a parada da busca para interromper uma soma que ainda precisa terminar.

Capacidade e quantidade utilizada

Um vetor pode ter espaço para cinco leituras e conter apenas três leituras válidas.

```c
int buffer[5] = {10, 20, 30, 0, 0};
int quantidade = 3;
if (quantidade < 5) {
    buffer[quantidade] = 40;
    quantidade++;
}
for (int i = 0; i < quantidade; i++) {
    printf("%d\n", buffer[i]);
}
```

- Capacidade continua 5; quantidade passa a 4.
- O espaço restante não deve entrar na média das leituras válidas.
- Inserção no meio e remoção exigem deslocar elementos; não são automáticas em um vetor C.

Matrizes: duas dimensões para organizar dados

Duas máquinas, três instantes de medição:

```
int temperatura[2][3] = {
    {24, 25, 26},
    {28, 29, 30}
};
```

	instante 0	instante 1	instante 2
máquina 0	24	25	26
máquina 1	28	29	30

`temperatura[1][2]` acessa a máquina 1 no instante 2: valor 30.

Índices de linha e coluna

- Em `int dados[2][3]`, há duas linhas e três colunas.
- Linhas válidas: 0 e 1; colunas válidas: 0, 1 e 2.
- Total de elementos: $2 \times 3 = 6$.
- Cada linha é um vetor; em C, as linhas ficam em sequência na memória.

`[0][0]`	`[0][1]`	`[0][2]`	`[1][0]`	`[1][1]`	`[1][2]`
24	25	26	28	29	30

Ambos os limites precisam ser respeitados

`dados[0][3]` é inválido, mesmo existindo uma linha seguinte. Não tente atravessar uma linha usando um índice de coluna fora do intervalo.

Percorrer matrizes com laços aninhados

```c
for (int linha = 0; linha < 2; linha++) {
    for (int coluna = 0; coluna < 3; coluna++) {
        printf("%d ", temperatura[linha][coluna]);
    }
    printf("\n");
}
```

Rastreio da ordem

`[0][0]` → `[0][1]` → `[0][2]` → `[1][0]` → `[1][1]` → `[1][2]`

- O laço externo escolhe a máquina.
- O laço interno percorre os três instantes dessa máquina.
- Ao mudar de linha, o índice de coluna começa novamente em zero.

Uma média para cada linha

```
for (int linha = 0; linha < 2; linha++) {
    int soma_linha = 0;
    for (int coluna = 0; coluna < 3; coluna++) {
        soma_linha += temperatura[linha][coluna];
    }
    double media = (double)soma_linha / 3;
    printf("Maquina %d: %.1f C\n", linha, media);
}
```

- Máquina 0: $(24 + 25 + 26)/3 = 25{,}0$.
- Máquina 1: $(28 + 29 + 30)/3 = 29{,}0$.
- `soma_linha` é reiniciada para cada máquina. Colocá-la antes do laço externo somaria também as linhas anteriores.

5.3 Strings: texto em um vetor de char

```c
char comando[6] = "LIGAR";
/* Equivalente a: {'L', 'I', 'G', 'A', 'R', '\0'} */
printf("Comando: %s\n", comando);
```

Índice	0	1	2	3	4	5
Conteúdo	L	I	G	A	R	\0

- Uma string C termina no primeiro caractere nulo, '\0'.
- Cinco letras precisam de pelo menos seis posições.
- %s espera uma string válida, com terminador acessível.

Terminador, caractere e capacidade

```c
char estado[10] = "ON";  /* 2 letras + terminador */
char letra = '0';        /* caractere do digito */
char fim = '\0';         /* valor zero: terminador */
estado[0] = '0';
estado[1] = 'K';
estado[2] = '\0';         /* string agora: "0K" */
```

- Aspas simples representam um caractere; aspas duplas, uma string literal.
- '0' e '\0' não representam o mesmo valor.
- `char texto[5] = "LIGAR";` não deixa espaço para o terminador e não deve ser usado como string.

strlen e sizeof respondem perguntas diferentes

```c
#include <string.h>
/* Dentro de main: */
char estado[10] = "ON";
printf("Comprimento: %zu\n", strlen(estado));
printf("Capacidade: %zu\n", sizeof estado);
```

Expressão	Significado neste exemplo	Resultado
`strlen(estado)`	Bytes anteriores ao terminador	2
`sizeof estado`	Bytes de todo este vetor	10

Os resultados usam `size_t`, impresso com `%zu`. `strlen` exige uma string terminada. Em UTF-8, uma letra acentuada pode ocupar mais de um byte; os comandos desta aula usam apenas ASCII.

strcmp: comparar o conteúdo de strings

```c
#include <string.h>
/* Dentro de main: */
char comando[16] = "LIGAR";
if (strcmp(comando, "LIGAR") == 0) {
    printf("Saida ativada\n");
} else {
    printf("Comando diferente\n");
}
```

- Retorno zero: textos iguais.
- Retorno negativo ou positivo: ordem lexicográfica; não suponha que será exatamente -1 ou 1.
- == não compara conteúdo de strings.
- Maiúsculas e minúsculas diferem: `LIGAR` e `ligar` não são iguais.

fgets: limitar a leitura à capacidade do vetor

```c
char comando[16];
if (fgets(comando, sizeof comando, stdin) == NULL) {
    printf("Nenhuma linha recebida\n");
    return 1;
}
printf("Recebido: %s", comando);
```

- Lê no máximo 15 caracteres e acrescenta `'\0'` em caso de sucesso.
- A quebra de linha também é armazenada se houver espaço.
- `NULL` indica que não foi possível obter a linha; não use o vetor como se a leitura tivesse funcionado.
- Uma linha maior que a capacidade fica parcialmente na entrada.

Remover a quebra de linha sem acessar antes do vetor

```
size_t tamanho = strlen(comando);
if (tamanho > 0 && comando[tamanho - 1] == '\n') {
    comando[tamanho - 1] = '\0';
}
```

- A verificação `tamanho > 0` vem antes do acesso à última posição.
- Trocar a quebra pelo terminador permite comparar `LIGAR` com o comando recebido pelo teclado.
- Uma linha vazia contém apenas `'\n'` antes dessa limpeza; depois, vira a string vazia.

Falta uma decisão

Se a quebra de linha não coube, precisamos tratar o restante da entrada antes de aceitar o comando.

Entrada longa: uma política explícita

Complemento ao teste da quebra de linha do slide anterior:

```
else {
    int extra = getchar();
    if (extra != '\n' && extra != EOF) {
        while ((extra = getchar()) != '\n' && extra != EOF) {
            /* descartar o restante da linha */
        }
        printf("Comando muito longo\n");
        return 1;
    }
}
```

- Se só faltou ler a quebra ou chegou o fim da entrada, o texto completo cabe e pode ser aceito.
- Se existe outro caractere, o texto foi cortado: descarte e rejeite.
- `getchar` retorna `int`, para representar caracteres e `EOF`.

Integração: um comando controla uma saída simulada

1. Ler uma linha com `fgets` e testar o retorno.
2. Remover a quebra de linha e rejeitar entrada longa.
3. Usar `strcmp` para reconhecer `LIGAR` e `DESLIGAR`.
4. Exibir o estado de saída ou indicar comando desconhecido.

Entrada	Resultado esperado
`LIGAR`	Saída simulada: 1
`DESLIGAR`	Saída simulada: 0
`ligar` ou linha vazia	Comando desconhecido
Mais de 15 caracteres	Comando muito longo

Código completo: `exemplos/03_comandos.c`. Evite misturar leituras `scanf` e `fgets` nesta atividade.

Prática guiada: analisar um lote de medições

Use o vetor `{18, 22, 22, 27, 31, 24}`.

1. Exiba cada valor junto de seu índice.
2. Calcule soma, média, mínimo e máximo.
3. Conte quantas leituras são maiores ou iguais a 25.
4. Busque a primeira ocorrência de 22.
5. Repita a busca com 99 e trate o caso ausente.

Critério de conclusão

Nenhum acesso fora de 0 a 5; média calculada em ponto flutuante; busca distingue índice zero de ausência; acumuladores inicializados antes de percorrer.

Extensão: altere o vetor para seis valores iguais e confira mínimo, máximo e contagem.

Exercícios de fixação

1. Em `int v[4] = {3, 6};`, informe todos os elementos e corrija um laço que usa `i <= 4`.
2. Para a matriz `{{10,20,30},{40,50,60}}`, obtenha a soma de cada coluna com laços aninhados.
3. Qual a capacidade mínima para guardar `PARAR`? Qual o resultado de `strlen("PARAR")`?
4. Por que `strcmp(a,b)==0` indica igualdade? Por que `a==b` não atende a essa tarefa?
5. Estenda o programa de comandos para aceitar `STATUS` e exibir "Saída inicial: 0". Preserve a validação de tamanho.

Entregue código, resultados e uma explicação dos limites. Confira `gabarito.md` depois de tentar.

Síntese e conexão com funções

- Vetores agrupam valores; matrizes acrescentam uma organização em linhas e colunas.
- Índices começam em zero e devem ficar dentro dos limites.
- Percursos permitem atualização, busca e estatísticas.
- Strings C dependem do terminador `'\0'`.
- `strlen`, `strcmp` e `fgets` têm papéis distintos: medir, comparar e ler.

Próxima aula

Como reutilizar o cálculo de média e a interpretação de comandos sem copiar blocos inteiros? Funções e modularização separam responsabilidades.

Material de apoio: exemplos C11, comandos de compilação no `README.md` e soluções em `gabarito.md`.